

Problem-Solving with Algorithms

Greedy Algorithm

Greedy algorithms aim to make the optimal choice at that given moment. Each step it chooses the optimal choice, without knowing the future. It attempts to find the globally optimal way to solve the entire problem using this method.

Why Are Greedy Algorithms Called Greedy?

We call algorithms *greedy* when they utilize the greedy property. The greedy property is:

At that exact moment in time, what is the optimal choice to make?

Greedy algorithms are greedy. They do not look into the future to decide the global optimal solution. They are only concerned with the optimal solution locally. This means that **the overall optimal solution may differ from the solution the algorithm chooses.**

They never look backwards at what they've done to see if they could optimize globally. This is the main difference between Greedy and Dynamic Programming.

Greedy algorithms don't guarantee solutions but are very time efficient. However, sometimes Greedy solutions give us the optimal solutions.

What Are Greedy Algorithms Used For?

Greedy algorithms are quick. A lot faster than the two other alternatives (Divide & Conquer, and Dynamic Programming). They're used because they're fast.

Sometimes, Greedy algorithms give the global optimal solution every time. Some of these algorithms are:

- Dijkstra's Algorithm
- Kruskal's algorithm
- Prim's algorithm
- Huffman trees

These algorithms are Greedy, and their Greedy solution gives the optimal solution.

How To Create a Greedy Algorithm

Your algorithm needs to follow this property:

At that exact moment in time, what is the optimal choice to make?

And that's it. There isn't much to it. Greedy algorithms are easier to code than Divide & Conquer or Dynamic Programming.

Dijkstra's Algorithm

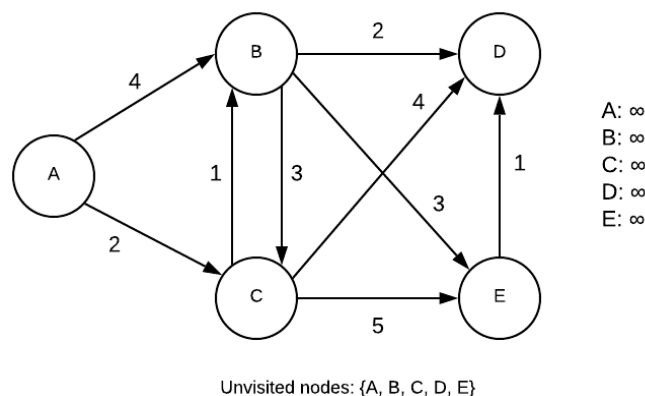
Dijkstra's algorithm finds the shortest path from a node to every other node in the graph. In this example, we'll be using a weighted directed graph. Each edge has a direction, and each edge has a weight.

Dijkstra's algorithm has many uses. It can be very useful within road networks where you need to find the fastest route to a place. We also use the algorithm for:

- IP Routing
- A* Algorithm
- Telephone networks

The algorithm follows these rules:

- i.) Every time we want to visit a new node, we will choose the node with the smallest known distance.
- ii.) Once we've moved to the node, we check each of its neighbouring nodes. We calculate the distance from the neighbouring nodes to the root nodes by summing the cost of the edges that lead to that new node.
- iii.) If the distance to a node is less than a known distance, we'll update the shortest distance.

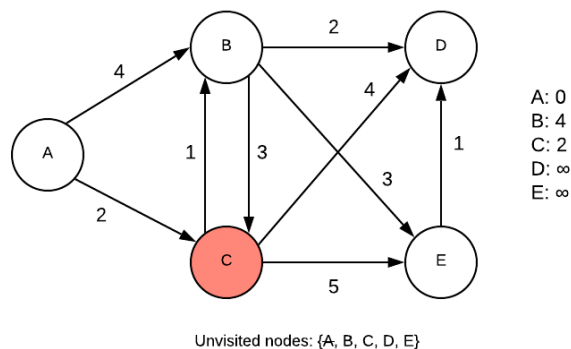


Our first step is to pick the starting node. Let's choose A. All the distances start at infinity, as we don't know their distance until we reach a node that knows the distance.

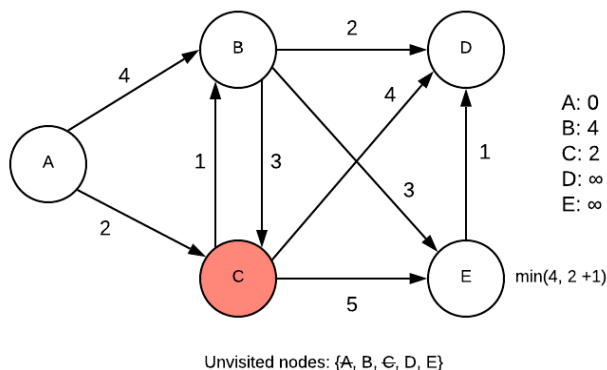
We mark off A on our unvisited nodes list. The distance from A to A is 0. The distance from A to B is 4. The distance from A to C is 2. We updated our distance listing on the right-hand side.

We pick the smallest edge where the vertex hasn't been chosen. The smallest edge is A → C, and we haven't chosen C yet. We visit C.

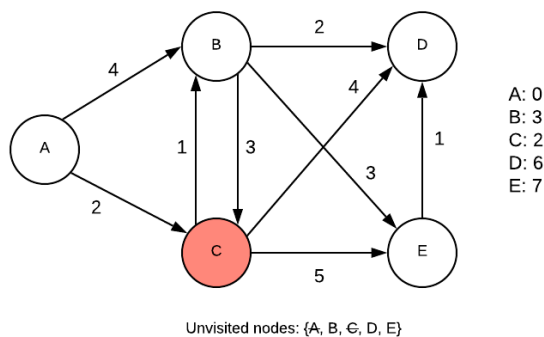
Notice how we're picking the smallest distance from our current node to a node we haven't visited yet. We're being greedy. Here, the greedy method is the global optimal solution.



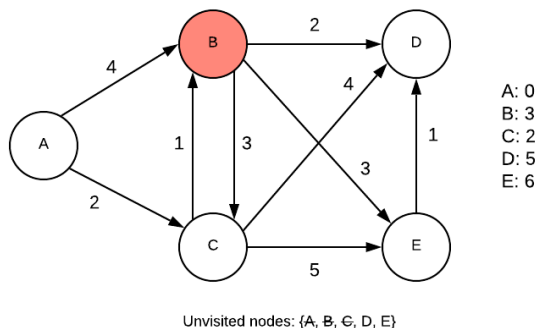
We can get to B from C. We now need to pick a minimum $\min(4, 2+1)=3$.



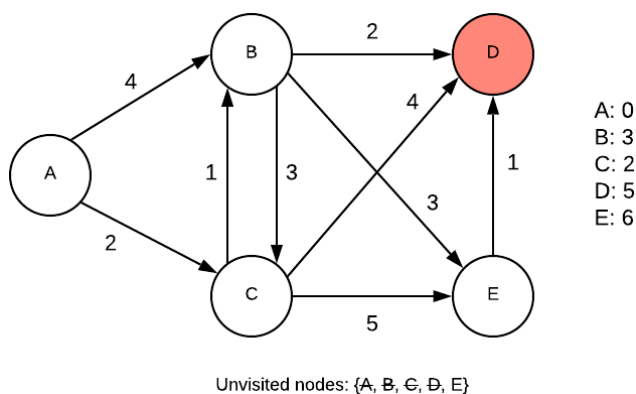
Since $A \rightarrow C \rightarrow B$ is smaller than $A \rightarrow B$, we update B with this information. We then add in the distances from the other nodes we can now reach.



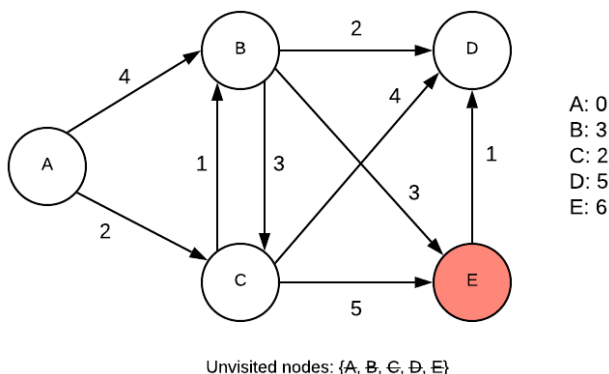
Our next smallest vertex with a node we haven't visited yet is B, with 3. We visit B.



We do the same for B. Then we pick the smallest vertex we haven't visited yet, D.



We don't update any of the distances this time. Our last node is then E.

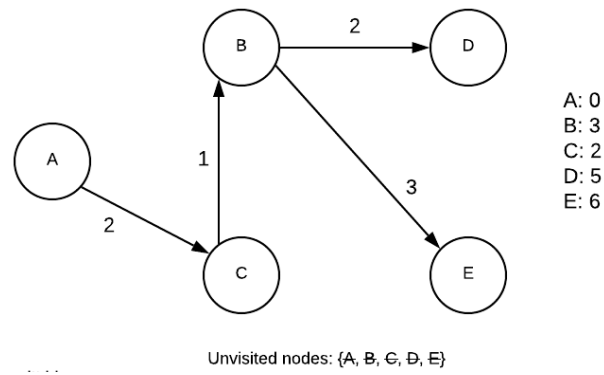


There are no updates again. To find the shortest path from A to the other nodes, we walk back through our graph.

We pick A first, C second, B third. If you need to create the shortest path from A to every other node as a graph, you can run this algorithm using a table on the right-hand side.

Dijkstra's Table		
Node	Distance from A	Previous node
A	0	N/A
B	3	C
C	2	A
D	5	B
E	6	B

Using this table it is easy to draw out the shortest distance from A to every other node in the graph:



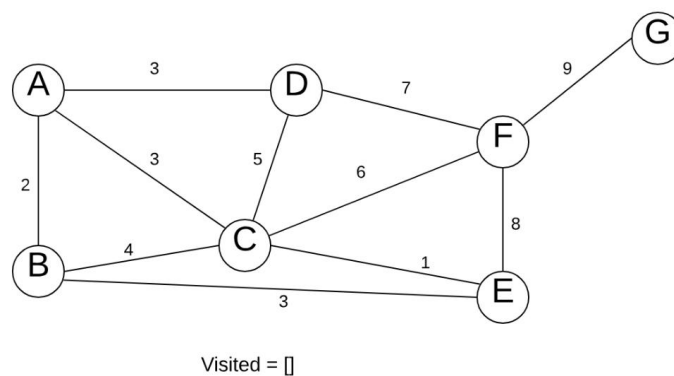
Minimum Spanning Trees Using Prim's Algorithm

Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a weighted undirected graph. It finds the optimal route from every node to every other node in the tree.

With a small change to Dijkstra's algorithm, we can build a new algorithm - Prim's algorithm!

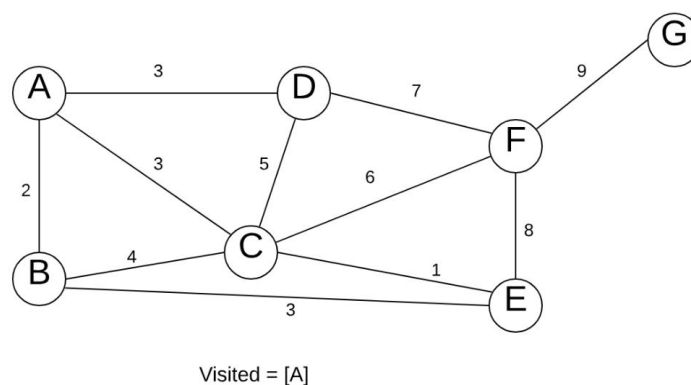
We informally describe the algorithm as:

- i.) Create a new tree with a single vertex (chosen randomly)
- ii.) Of all the edges not yet in the new tree, find the minimum weighted edge and transfer it to the new tree
- iii.) Repeat step 2 until all vertices are in the tree



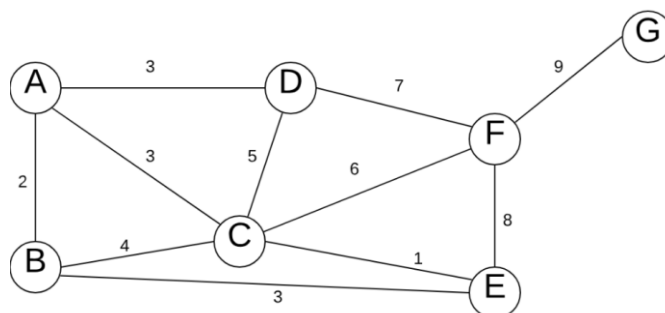
We have this graph.

Our next step is to pick an arbitrary node.



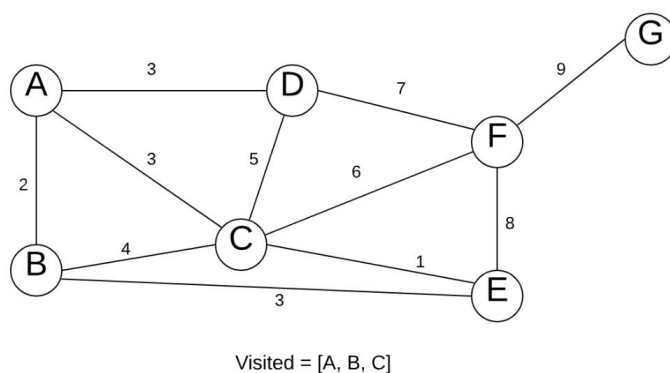
We pick the node A. We then examine all the edges connecting A to other vertices. Prim's algorithm is greedy. That means **it picks the shortest edge that connects to an unvisited vertex.**

In our example, it picks B.

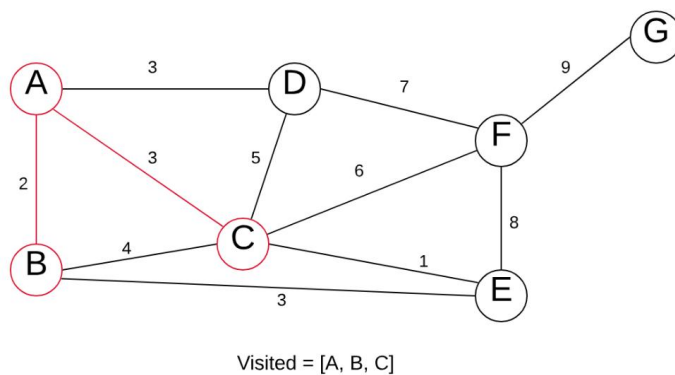


We now look at all nodes reachable from A **and** B. This is the distinction between Dijkstra's and Prim's. With Dijkstra's, we're looking for a path from 1 node to a certain other node (nodes that have not been visited). With Prim's, we want the minimum spanning tree.

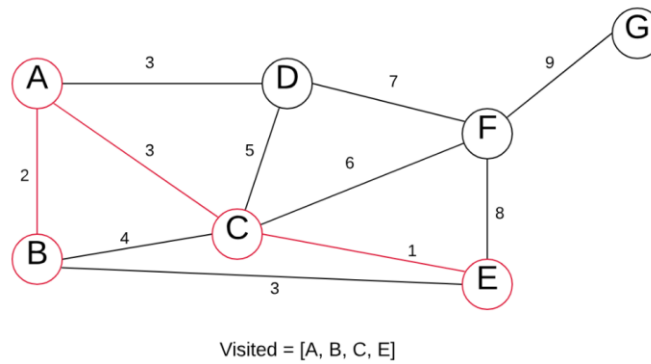
We have 3 edges with equal weights of 3. We pick 1 randomly.



It is helpful to highlight our graph as we go along because it makes it easier to create the minimum spanning tree.

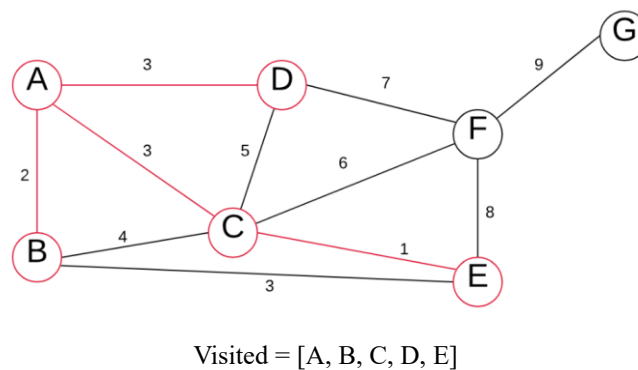


Now we look at all edges of A, B, and C. The shortest edge is C > E with a weight of 1.

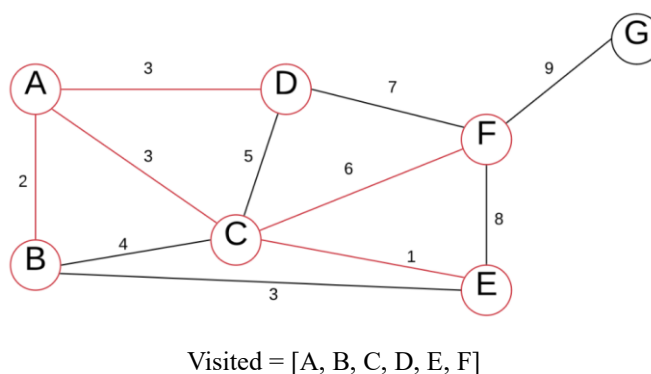


And we repeat:

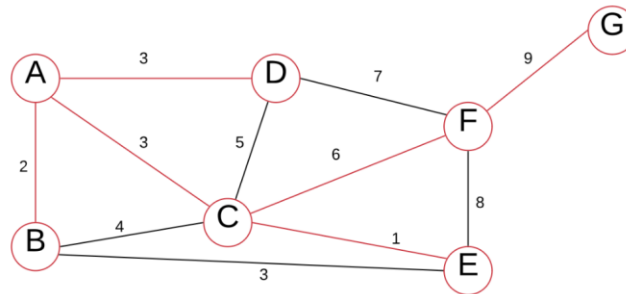
Now we look at all edges of A, B, C and E. The shortest edges are A > D and B > E with a weight of 3.



Now we look at all edges of B, C, D and E. The shortest edge is B > E with a weight of 3. However, both vertices are always in our VISITED list. Meaning we do not pick this edge. We instead choose C > F, as we have not visited F

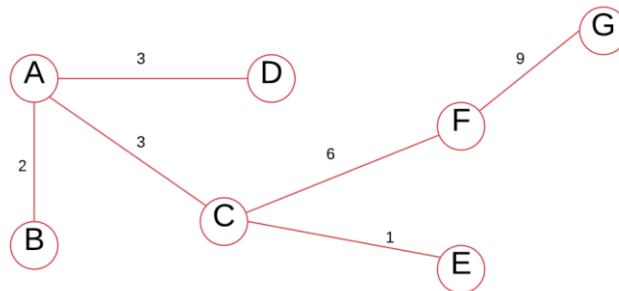


The only node left is G, so we visit it.



Visited = [A, B, C, D, E, F, G]

Note that if the edge weights are distinct, the minimum spanning tree is unique.



We can add the edge weights to get the minimum spanning tree's total edge weight:

$$2 + 3 + 3 + 1 + 6 + 9 = 24$$

Fractional Knapsack Problem Using Greedy Algorithm

Imagine you are a thief. You break into John Maina's house. The house has a number of valuable items.

You brought with you a bag - a knapsack. This bag can carry a maximum of 50 kgs. You happen to have a listing of JM's items, from some insurance paper. The items read as:

JM's Items		
Item	Value in Kshs.	Weight in Kgs
Item 1	100,000/=	20
Item 2	120,000/=	30
Item 3	60,000/=	10

You want to leave the house with as much value as possible in your knapsack (bure kabisa!)

The first step to solving the fractional knapsack problem is to calculate **value / weight** for each item.

JM's Items			
Item	Value in Kshs.	Weight in Kgs	Value / Weight
Item 1	100,000/=	20	5,000
Item 2	120,000/=	30	4,000
Item 3	60,000/=	10	6,000

And now we greedily select the largest ones. To do this, we sort them according to **value / weight** in descending order

JM's Items			
Item	Value in Kshs.	Weight in Kgs	Value / Weight
Item 3	60,000/=	10	6,000
Item 1	100,000/=	20	5,000
Item 2	120,000/=	30	4,000

We select Item 3

knapsack value = 60,000

knapsack total weight = 10 (out of 50)

Next, we select Item 1

knapsack value = 160,000

knapsack total weight = 30 (out of 50)

Finally, we need to add Item 2.

But if we add item 2, our total weight will come to 80 (maximum is 50!).

In the fractional knapsack problem, we can cut items up to take fractions of them. We have a weight of 20 left in the bag. Item 2 has a weight 30. We calculate the ratio of:

weight of knapsack left / weight of item

And then multiply this ratio by the value of the item to get how much value of that item we can take.

$$20 / 30 * 120,000 = 80,000$$

$$\text{knapsack value} = 240,000$$

$$\text{knapsack total weight} = 50 \text{ (out of 50)}$$

The greedy algorithm can optimally solve the fractional knapsack problem, but it cannot optimally solve the $\{0, 1\}$ knapsack problem. In this problem instead of taking a fraction of an item, you either take it $\{1\}$ or you don't $\{0\}$. To solve this, you need to use Dynamic Programming.

Summary Using Arrays

Given two arrays, **val[]** and **wt[]**, representing the values and weights of item respectively, and an integer **capacity** representing the maximum weight a knapsack can hold, we have to determine the maximum total value that can be achieved by putting the items in the knapsack without exceeding its capacity.

Items can also be taken in fractional parts if required.

Example:

Input: $\text{val[]} = [100000, 120000, 60000]$, $\text{wt[]} = [20, 30, 10]$, $\text{capacity} = 50$

Output: 240000

Explanation: We will take the items of weight 10kg and 20kg and $2/3$ fraction of 30kg. Hence total value will be $60000 + 100000 + (2/3) * 120000 = 240000$.

Why naive approach will fail

Case 1: Picking the items with smaller weights first

Example: $\text{val[]} = [10, 10, 10, 100]$, $\text{wt[]} = [10, 10, 10, 30]$, $\text{capacity} = 30$

- If we start picking smaller weights, we can pick the three items of weight 10 → total value = $10+10+10 = 30$.
- But the optimal choice is to take the last item (value 100, weight 30) → total value = 100.

So, choosing smaller weights first fails here.

Case 2: Picking items with larger value first

Example: $\text{val[]} = [10, 10, 10, 20]$, $\text{wt[]} = [10, 10, 10, 30]$, $\text{capacity} = 30$

- If we start picking higher values, we might choose the last item (value 20, weight 30) → total value = 20.
- But the better choice is to take the three items of weight 10 each → total value = $10+10+10 = 30$.

So, choosing higher values first also fails.

[Approach] Selecting Items by value/weight Ratio

*The idea is to always pick items greedily based on their **value-to-weight ratio**. Take the item with the highest ratio first, then the next highest, and so on, until the knapsack is full. If any item doesn't fully fit, then take its fractional part according to the remaining capacity.*

Steps to solve the problem:

- i.) Calculate the ratio (value/weight) for each item.
- ii.) Sort all the items in decreasing order of the ratio.
- iii.) Iterate through items:
 - if the current item fully fits, add its full value and decrease capacity otherwise, take the fractional part that fits and add proportional value.
- iv.) Stop once the capacity becomes zero.